

1 Orthogonal Matrices

(a) If U and V are orthogonal, then $U^T U = I$ and $V^T V = I$. Hence

$$(UV)^T(UV) = V^T U^T UV = V^T V = I$$

Thus UV has orthonormal columns. Since U and V are square, UV is square, so UV is orthogonal.

(b) Since U is orthogonal,

$$U^T U = I$$

Therefore $U^{-1} = U^T$, and also $UU^T = I$. Then

$$(U^{-1})^T U^{-1} = (U^T)^T U^T = UU^T = I$$

Thus U^{-1} is orthogonal.

(c) Let

$$U = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

The first column of U has norm one, so for some angle θ ,

$$\begin{bmatrix} a \\ c \end{bmatrix} = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}$$

The second column must be a unit vector orthogonal to the first column. In $\mathbb{R}^2$, there are only two choices:

$$\begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}, \quad \begin{bmatrix} \sin \theta \\ -\cos \theta \end{bmatrix}$$

Therefore every 2×2 orthogonal matrix has one of the two forms

$$U = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

or

$$U = \begin{bmatrix} \cos \theta & \sin \theta \\ \sin \theta & -\cos \theta \end{bmatrix}$$

The first matrix has determinant $+1$ and is a rotation. The second matrix has determinant -1 and is a reflection. Therefore, to decide which case a given orthogonal U is in, compute

$$\det U$$

If $\det U = 1$, it is a rotation. If $\det U = -1$, it is a reflection.

2 Groups of Equivalent Statements

a, e, i

b, g, k

c

d, h

f, j

3 Approximate Inductance Formula

The model is

$$\hat{L} = \alpha n^{\beta_1} w^{\beta_2} d^{\beta_3} D^{\beta_4}$$

Taking logs gives a linear least-squares problem:

$$\log \hat{L}_i = \log \alpha + \beta_1 \log n_i + \beta_2 \log w_i + \beta_3 \log d_i + \beta_4 \log D_i$$

Define

$$\theta = \begin{bmatrix} \log \alpha \\ \beta_1 \\ \beta_2 \\ \beta_3 \\ \beta_4 \end{bmatrix}, \quad X_i = [1 \quad \log n_i \quad \log w_i \quad \log d_i \quad \log D_i]$$

and $z_i = \log L_i$. The model can be fitted by minimizing

$$\|X\theta - z\|_2^2$$

This minimizes the sum of squared log-errors, which is a natural criterion here because the model is multiplicative and the final error is relative to the true inductance.

```
alpha = 0.00070725326
beta1 = 1.3793645
beta2 = -0.48060387
beta3 = 0.27561595
beta4 = 1.2131716
average percentage error = 6.7700%
```

3.1 Code

```
1 using LinearAlgebra
2 using Statistics
3
4 include(joinpath(@__DIR__, "readclassjson.jl"))
5
6 data = readclassjson(joinpath(@__DIR__, "inductor_data.json"))
7 nturns = data["n"]
8 w = data["w"]
9 d = data["d"]
10 D = data["D"]
11 L = data["L"]
12
13 X = hcat(ones(length(L)), log.(nturns), log.(w), log.(d), log.(D))
14 theta = X \ log.(L)
15
16 alpha = exp(theta[1])
17 beta = theta[2:end]
18
```

```
19 | Lhat = exp.(X * theta)
20 | pct_error = 100 .* abs.(Lhat .- L) ./ L
21 |
22 | average_error = mean(pct_error)
```

4 Image Reconstruction from Line Integrals

Let $x \in \mathbb{R}^{n^2}$ be the vector of unknown pixel densities, and $A \in \mathbb{R}^{N \times n^2}$ a measurement matrix:

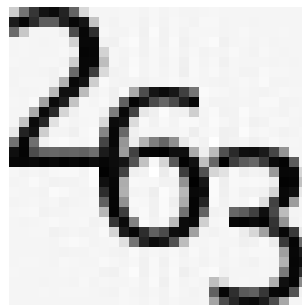
$$A_{ji} = l_{ij}$$

Then the measurements have the form

$$y = Ax + v$$

Because the measurements are noisy, x can be estimated using least squares:

$$\hat{x} = \arg \min_x \|Ax - y\|_2^2$$



4.1 Code

```

1 using LinearAlgebra
2 using Plots
3
4 include(joinpath(@__DIR__, "readclassjson.jl"))
5
6 data = readclassjson(joinpath(@__DIR__, "tomo_data.json"))
7 N = data["N"]
8 npixels = data["npixels"]
9 y = data["y"]
10 line_pixel_lengths = data["line_pixel_lengths"]
11
12 A = line_pixel_lengths'
13 xhat = A \ y
14 residual = A * xhat - y
15
16 image = reshape(xhat, npixels, npixels)
17
18 heatmap(image; yflip=true, aspect_ratio=:equal,
19         color=:gist_gray, cbar=:none, framestyle=:none)

```

5 Curve Smoothing

(a) Define $D \in \mathbb{R}^{(n-2) \times n}$ by

$$(Dg)_i = \frac{g_{i+2} - 2g_{i+1} + g_i}{(1/n)^2}, \quad i = 1, \dots, n-2$$

Then

$$d = \frac{1}{n} \|f - g\|_2^2, \quad c = \frac{1}{n-2} \|Dg\|_2^2$$

For a fixed $\mu \geq 0$, minimize

$$d + \mu c = \frac{1}{n} \|f - g\|_2^2 + \frac{\mu}{n-2} \|Dg\|_2^2$$

Setting the gradient equal to zero gives

$$\begin{aligned} \nabla_g(d + \mu c) &= \nabla_g \left(\frac{1}{n} (f - g)^T (f - g) + \frac{\mu}{n-2} g^T D^T D g \right) = 0 \\ &= \frac{2}{n} (g - f) + \frac{2\mu}{n-2} D^T D g = 0 \\ &= \frac{2}{n} g + \frac{2\mu}{n-2} D^T D g = \frac{2}{n} f \\ &= \left(\frac{1}{n} I + \frac{\mu}{n-2} D^T D \right) g = \frac{1}{n} f \end{aligned}$$

The matrix on the left is positive definite for every $\mu \geq 0$, because $(1/n)I$ is positive definite and $(\mu/(n-2))D^T D$ is positive semidefinite. Therefore the method always gives a unique solution:

$$g^* = \left(\frac{1}{n} I + \frac{\mu}{n-2} D^T D \right)^{-1} \frac{1}{n} f$$

For $\mu = 0$, the solution is

$$g^* = f$$

For $\mu \rightarrow \infty$, curvature dominates (more weight to minimize c), so the limiting solution must satisfy

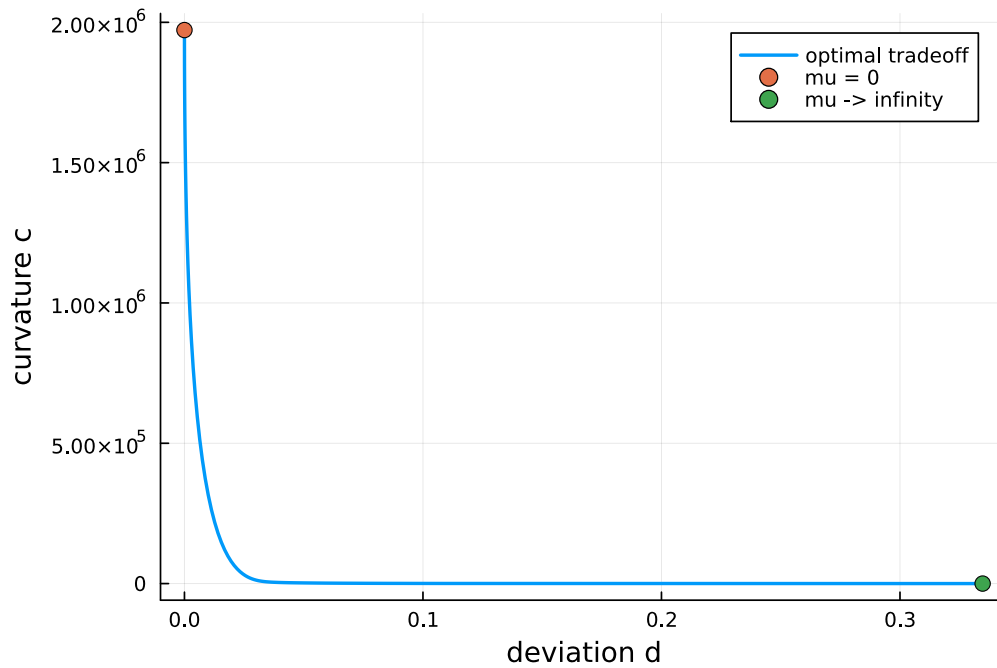
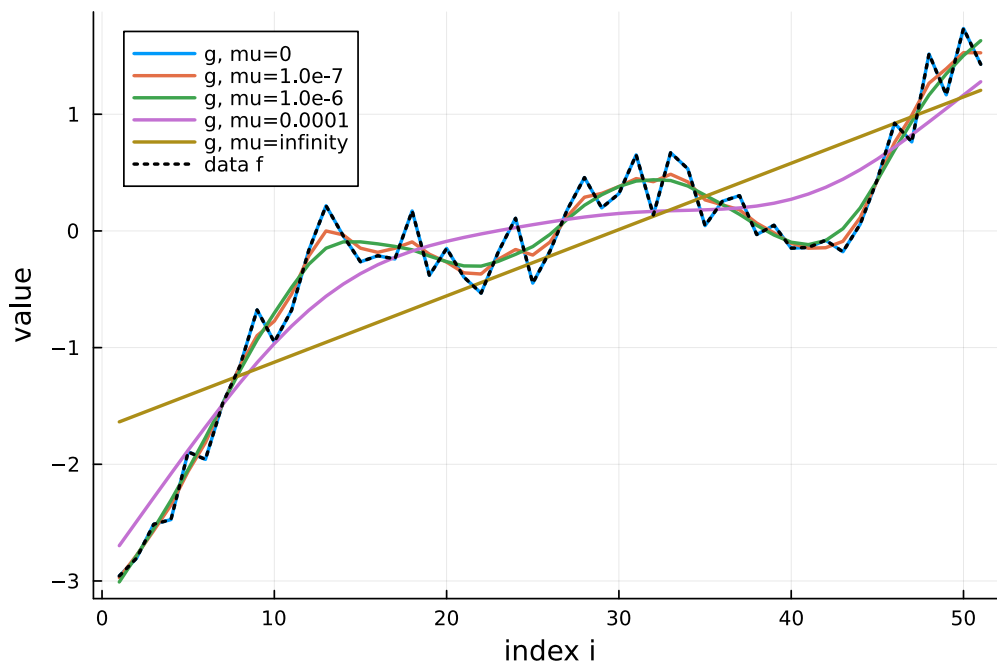
$$Dg = 0$$

The nullspace of D is the set of affine sequences. Therefore the $\mu \rightarrow \infty$ solution is the least-squares projection of f onto

$$\text{span} \left\{ \mathbf{1}, \begin{bmatrix} 1/n \\ 2/n \\ \vdots \\ n/n \end{bmatrix} \right\}$$

(b)

```
mu = 0: d = 3.6252799e-34, c = 1972426.4
mu = 1.0e-07: d = 0.021276065, c = 60135.072
mu = 1.0e-06: d = 0.033252238, c = 7361.6337
mu = 1.0e-04: d = 0.10846577, c = 431.20149
mu -> infinity: d = 0.33465062, c = 5.6272713e-25
```

Figure 1: Optimal trade-off between d and c Figure 2: Original data and smoothed curves for values of μ

5.1 Code

```

1 using LinearAlgebra
2 using Plots

```

```
3
4 include(joinpath(@__DIR__, "readclassjson.jl"))
5
6 function second_difference_matrix(n)
7     D = zeros(n - 2, n)
8     scale = n^2
9     for i in 2:(n - 1)
10         row = i - 1
11         D[row, i - 1] = scale
12         D[row, i] = -2scale
13         D[row, i + 1] = scale
14     end
15     return D
16 end
17
18 function smooth_curve(f, D, mu)
19     n = length(f)
20     if isinf(mu)
21         T = hcat(ones(n), collect(1:n) ./ n)
22         return T * (T \ f)
23     end
24
25     H = Matrix{Float64}(I, n, n) / n + mu * (D' * D) / (n - 2)
26     return H \ (f / n)
27 end
28
29 deviation(f, g) = sum((f .- g).^2) / length(f)
30 curvature(D, g) = sum((D * g).^2) / size(D, 1)
31
32 data = readclassjson(joinpath(@__DIR__, "curve_smoothing.json"))
33 n = data["n"]
34 f = data["f"]
35 D = second_difference_matrix(n)
36
37 mu_values = 10.0 .^ range(-13, -4, length=120)
38 g_values = [smooth_curve(f, D, mu) for mu in mu_values]
39 d_values = [deviation(f, g) for g in g_values]
40 c_values = [curvature(D, g) for g in g_values]
41
42 g0 = smooth_curve(f, D, 0.0)
43 ginf = smooth_curve(f, D, Inf)
44 d_plot = vcat(deviation(f, g0), d_values, deviation(f, ginf))
45 c_plot = vcat(curvature(D, g0), c_values, curvature(D, ginf))
46
47 plot(d_plot, c_plot, label="optimal tradeoff")
48 scatter!([d_plot[1], d_plot[end]], [c_plot[1], c_plot[end]],
49         label=["mu = 0" "mu -> infinity"])
50
51 selected_mu = [1e-7, 1e-6, 1e-4]
52 selected_g = [smooth_curve(f, D, mu) for mu in selected_mu]
53
54 xs = collect(1:n)
55 curve_plot = plot(xs, g0, label="g, mu=0",
56                 xlabel="index i", ylabel="value")
```



```
57 for (mu, g) in zip(selected_mu, selected_g)
58     plot!(curve_plot, xs, g, label="g, mu=$(mu)")
59 end
60 plot!(curve_plot, xs, ginf, label="g, mu=infinity")
61 plot!(curve_plot, xs, f, label="data f",
62     color=:black, linestyle=:dot)
```